

Home

Funcionalidad

Clases

Restricciones

Por:

Romero Yahir

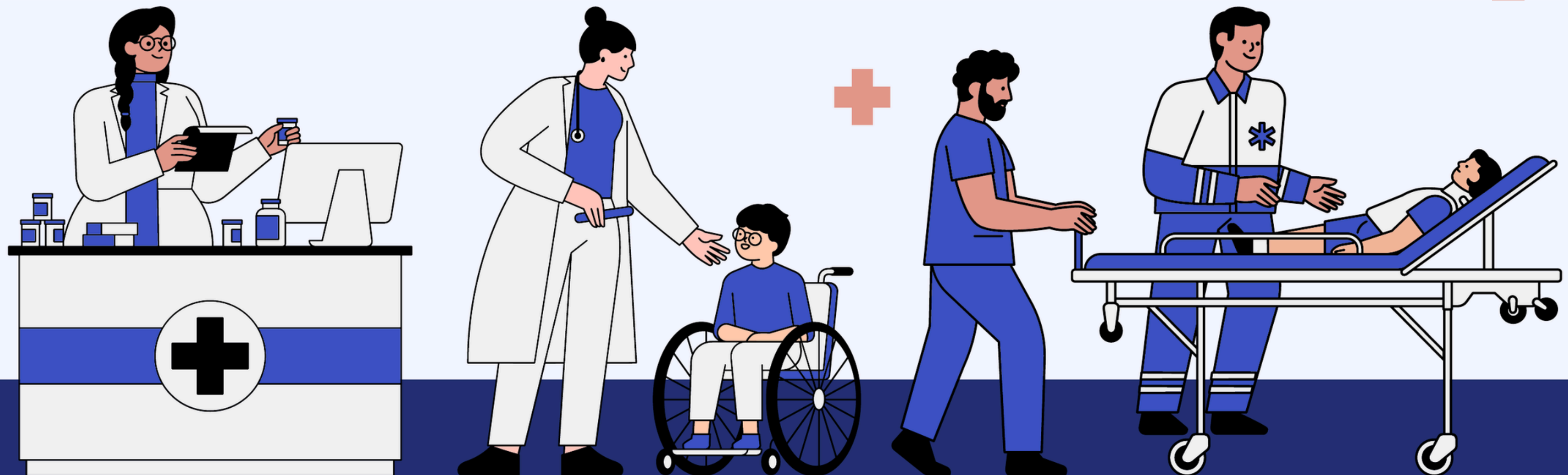
Carlos Julius

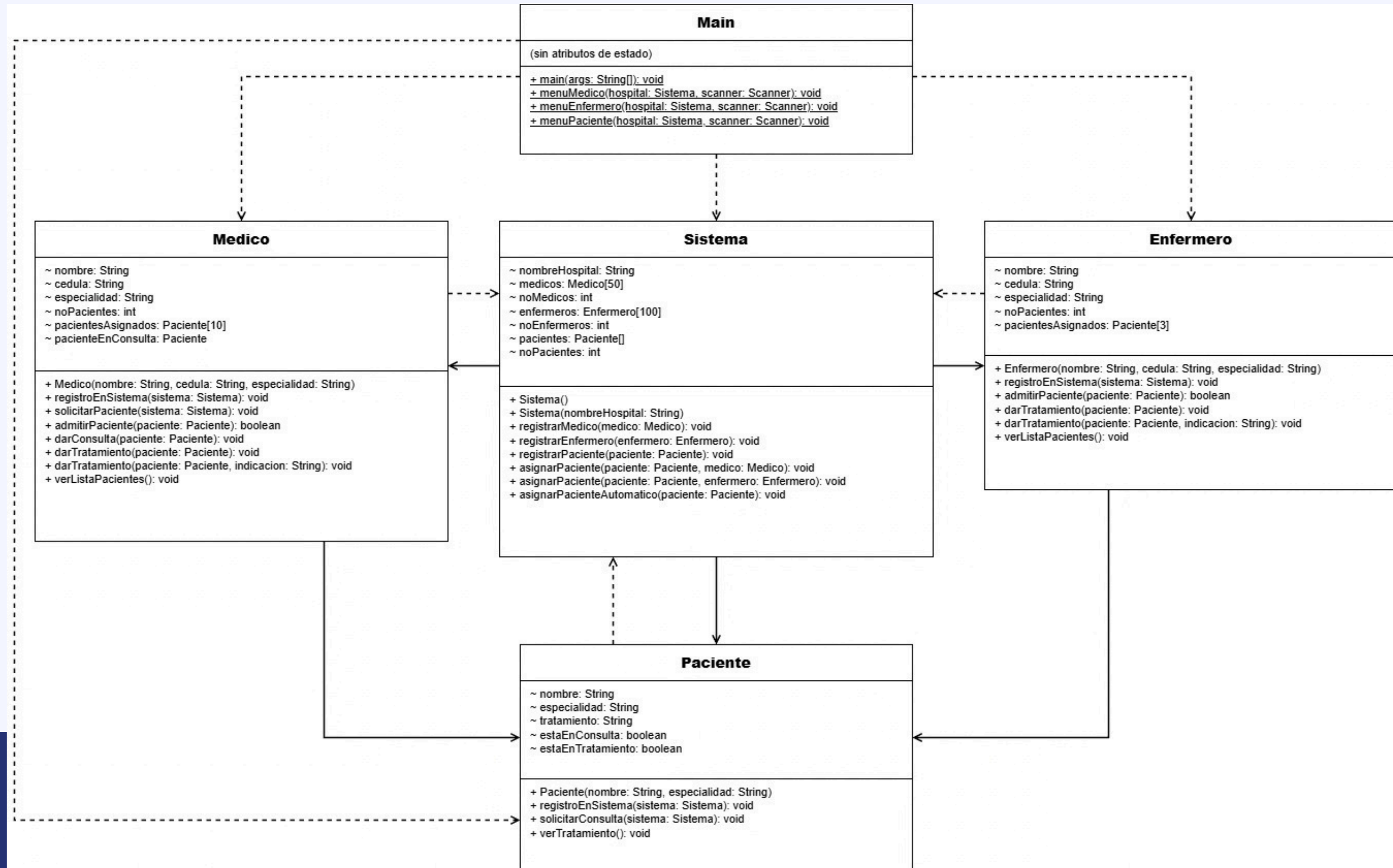
Porto Emanuel

Sulvarán Iván

Zayas Diego

Simulador de sistema hospitalario





Roles/Clases

Arreglos con contador

50 médicos y 100 enfermeros. Los pacientes inician en 10 y, al llenarse, se crea un arreglo del doble, se copian los registrados y se reemplaza el viejo.

Validación de especialidad

Antes de asignar, compara con equalsIgnoreCase. Solo si coincide, pide la admisión al médico o al enfermero, que revisan su cupo.

Métodos sobrecargados

asignarPaciente(Paciente, Medico) y
asignarPaciente(Paciente, Enfermero). El constructor también: con o sin nombre de hospital.

Asignación automática

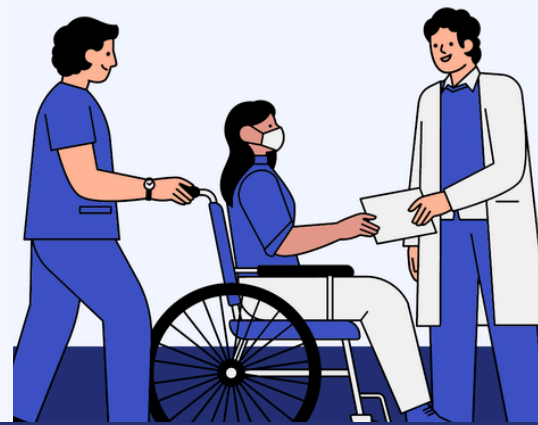
asignarPacienteAutomatico recorre los médicos y elige el primero de la especialidad del paciente con cupo disponible.

Sistema



Sistema

- ~ nombreHospital: String
 - ~ medicos: Medico[50]
 - ~ noMedicos: int
 - ~ enfermeros: Enfermero[100]
 - ~ noEnfermeros: int
 - ~ pacientes: Paciente[]
 - ~ noPacientes: int
-
- + Sistema()
 - + Sistema(nombreHospital: String)
 - + registrarMedico(medico: Medico): void
 - + registrarEnfermero(enfermero: Enfermero): void
 - + registrarPaciente(paciente: Paciente): void
 - + asignarPaciente(paciente: Paciente, medico: Medico): void
 - + asignarPaciente(paciente: Paciente, enfermero: Enfermero): void
 - + asignarPacienteAutomatico(paciente: Paciente): void



Medico

- ~ nombre: String
- ~ cedula: String
- ~ especialidad: String
- ~ noPacientes: int
- ~ pacientesAsignados: Paciente[10]
- ~ pacienteEnConsulta: Paciente

- + Medico(nombre, cedula, especialidad)
- + registroEnSistema(sistema: Sistema): void
- + solicitarPaciente(sistema: Sistema): void
- + admitirPaciente(paciente: Paciente): boolean
- + darConsulta(paciente: Paciente): void
- + darTratamiento(paciente: Paciente): void
- + darTratamiento(paciente, indicacion: String): void
- + verListaPacientes(): void

Médico

Cupo de 10 pacientes

pacientesAsignados[10] y noPacientes. admitirPaciente rechaza al paciente 11 y devuelve false.

Una consulta a la vez

pacienteEnConsulta guarda al paciente activo. darConsulta rechaza a otro mientras haya uno en consulta.

Receta

darTratamiento está sobrecargado: con o sin indicación. Al recetar, el médico queda libre y el paciente pasa a tratamiento.

Lista de Z a A

verListaPacientes copia el arreglo y aplica burbuja comparando charAt(0): intercambia si la letra izquierda es menor.

Enfermero



Cupo de 3 pacientes

pacientesAsignados[3]. admitirPaciente rechaza al paciente 4.

No da consultas

No tiene pacienteEnConsulta: su función exclusiva es administrar tratamientos.

Administra el tratamiento

darTratamiento muestra la indicación que recetó el médico. La versión sobrecargada permite escribir una indicación propia.

Misma lista Z a A

Mismo ordenamiento burbuja que el médico, sobre una copia del arreglo.

Enfermero

~ nombre: String
~ cedula: String
~ especialidad: String
~ noPacientes: int
~ pacientesAsignados: Paciente[3]

+ Enfermero(nombre, cedula, especialidad)
+ registroEnSistema(sistema: Sistema): void
+ admitirPaciente(paciente: Paciente): boolean
+ darTratamiento(paciente: Paciente): void
+ darTratamiento(paciente, indicacion: String): void
+ verListaPacientes(): void

Paciente

En espera

consulta = false
tratamiento = false

En consulta

estaEnConsulta
= true

**En
tratamiento**

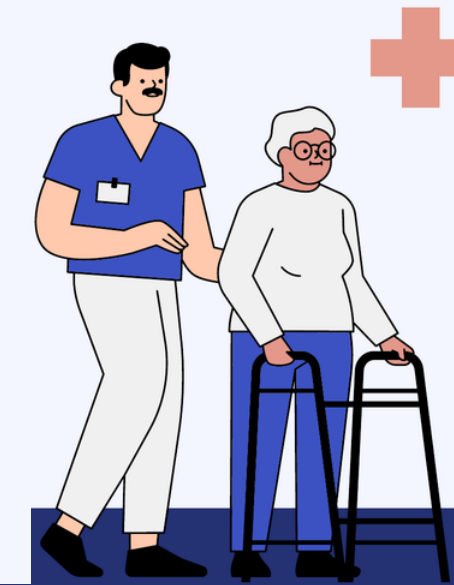
estaEnTratamiento
= true

Registro y solicitud

registroEnSistema y solicitarConsulta delegan en Sistema, que valida y asigna.

Mecanismo de Estado

El paciente nace en espera con ambos booleanos en falso. Medico.darConsulta activa el primero, y darTratamiento (médico o enfermero) activa el segundo y apaga el otro



Paciente

~ nombre: String
~ especialidad: String
~ tratamiento: String
~ estaEnConsulta: boolean
~ estaEnTratamiento: boolean

+ Paciente(nombre: String, especialidad: String)
+ registroEnSistema(sistema: Sistema): void
+ solicitarConsulta(sistema: Sistema): void
+ verTratamiento(): void

Main

(sin atributos de estado)

```
+ main(args: String[]): void  
+ menuMedico(hospital: Sistema, scanner: Scanner): void  
+ menuEnfermero(hospital: Sistema, scanner: Scanner): void  
+ menuPaciente(hospital: Sistema, scanner: Scanner): void
```

Menú principal: do-while con switch

1 Médico

2 Enfermero

3 Paciente

4 Salir

Un Scanner y un Sistema

Se crean una sola vez y se pasan a los tres menús, así todos trabajan sobre el mismo hospital.

Patrón de cada opción

Listar con for, leer el número y restar 1, validar el rango y llamar al método.

Home

Funcionalidad

Clases

THE END

